阶段 1 | Decoder-only 生成与 nanoGPT

> 本阶段回答一个核心问题：一个 decoder-only 语言模型怎样把一段文本变成下一个 token 的概率，并在推理时逐个 token 生成后续文本？

本阶段的位置

阶段 0 建立了 token、模型、损失、训练/推理和 PyTorch 的共同语言。本阶段把这些概念接成一个完整的生成系统：

```
原始文本
  ↓ tokenizer
token ID 序列
  ↓ 切成输入 x 与右移目标 y
decoder-only Transformer
  ↓ 每个位置的 logits
最后一个位置的 logits
  ↓ softmax / 温度 / top-k / top-p
选出一个新 token
  ↓ 追加到上下文并重复
生成文本
```

这里的重点是**数据流、张量形状和因果约束**。代码只做小规模可观察实验，不追求训练出有用的大语言模型。

完成后应达到的能力

1. 能解释字符级、词级和子词 tokenizer 的输入、输出、词表与未知词问题。
2. 能从联合概率分解写出自回归语言模型的训练目标，并说明输入与目标为什么错开一位。
3. 能画出 decoder-only Transformer block 的数据流：embedding、位置表示、因果自注意力、MLP、残差、归一化和输出 logits。
4. 能按张量形状解释 `B、T、C、V`，并判断 `CrossEntropyLoss` 的输入/目标是否摆放正确。
5. 能手写一个最小生成循环，说明 greedy、temperature、top-k、top-p 的差别和边界条件。
6. 能解释 KV cache 为什么只缓存历史的 K/V、为什么当前步只需要一个 Q，以及它节省了什么计算。
7. 能按入口、数据、模型、训练、采样的顺序阅读 nanoGPT，而不是从整个仓库逐文件漫游。
8. 能运行一个极小字符级 decoder-only 模型，记录训练损失、生成样例和失败原因。

建议时间与学习顺序

建议用 7 次学习、总计约 9–14 小时，可在 1–2 周完成。每次先读对应学案，再运行一个观察程序，最后用自己的话写出“输入 → 形状 → 运算 → 输出”。

次序	内容	建议时间	完成标志
1	tokenizer、词表与数据切片	60–100 分钟	能 encode/decode，并写出一对错开一位的 x/y

次序	内容	建议时间	完成标志
2	decoder-only 概率与训练目标	75–110 分钟	能解释 logits、softmax、交叉熵和 next-token prediction
3	因果自注意力与 decoder block	100–150 分钟	能读懂形状为 `(B,T,C)` 的 attention 数据流
4	生成循环与采样	75–120 分钟	能手写停止条件，并比较四种采样行为
5	KV cache	75–120 分钟	能解释 full recompute 与 incremental cache 的差异，并通过数值对照
6	nanoGPT 代码阅读	120–180 分钟	能沿一次训练和一次采样调用画出调用图
7	极小模型实验与复盘	120–180 分钟	能改一个变量、记录结果、解释生成质量的边界

推荐操作顺序

1. 读 01_文本与Tokenizer学案，运行 `代码/01_toy_tokenizer.py`。
2. 读 02_DecoderOnlyTransformer学案，运行 `代码/02_causal_attention.py`。
3. 先不看答案，完成 `练习/01_补全_tokenizer.py`；再对照 `答案/01_tokenizer参考实现.py`。
4. 读 03_生成循环与采样学案，运行 `代码/03_sampling_strategies.py`，然后完成 `练习/02_补全生成循环.py`。
5. 读 04_KVCache与推理效率学案，运行 `代码/04_kv_cache_compare.py`，再完成 `练习/03_补全KVCache.py`。
6. 按 05_nanoGPT阅读与实验学案 阅读本地 `参考资料/nanoGPT`，第一遍只追踪一次端到端调用。
7. 运行 `代码/05_tiny_decoder_lm.py`，最后闭卷完成 06_阶段自测与复盘。

代码学习方法

每个程序先只回答四件事：

- 输入张量是什么，形状是什么？
- 当前运算为什么允许看到这些位置，不能看到哪些位置？
- 输出是数值、概率、token ID 还是文本？
- 如果结果异常，先检查哪一个中间量？

运行时建议记录：随机种子、设备、词表大小、上下文长度、模型参数量、训练步数、训练/验证损失、生成提示和生成结果。生成样例必须和配置一起保存，否则无法比较实验。

本阶段的边界

本阶段不要求：

- 训练大模型或下载任何预训练权重；
- 实现工业级 BPE/SentencePiece tokenizer；
- 复现 nanoGPT 的 GPU 性能、混合精度或分布式训练；

- 证明一次生成样例代表模型具有可靠语言能力；
- 处理 beam search、speculative decoding、paged attention 等进阶推理系统。

字符级 tokenizer 是为了看清机制，不是生产方案。nanoGPT 仓库中的 Shakespeare 训练脚本主要作为代码阅读材料；本阶段的可运行实验使用本目录的短文本，避免下载大数据集。

运行环境

从项目根目录 `D:\TWBZX\python_study\agent` 执行：

```
& '.\.venv\Scripts\python.exe' '.\阶段1_decoder_only生成与nanoGPT\代码
\01_toy_tokenizer.py'
& '.\.venv\Scripts\python.exe' '.\阶段1_decoder_only生成与nanoGPT\代码
\02_causal_attention.py'
& '.\.venv\Scripts\python.exe' '.\阶段1_decoder_only生成与nanoGPT\代码
\03_sampling_strategies.py'
& '.\.venv\Scripts\python.exe' '.\阶段1_decoder_only生成与nanoGPT\代码
\04_kv_cache_compare.py'
& '.\.venv\Scripts\python.exe' '.\阶段1_decoder_only生成与nanoGPT\代码
\05_tiny_decoder_lm.py'
```

所有代码都可以在 CPU 上运行。若想切换设备，只对 `05_tiny_decoder_lm.py` 设置环境变量 `STAGE1_DEVICE=cuda`；先确认 `torch.cuda.is_available()`，不要为了追求速度改变模型规模。

参考资料

资料已经下载到 参考资料；具体版本和阅读重点见 参考资料/阅读说明。建议顺序是：

1. 先读本阶段学案，再看 Transformer 原论文中与 scaled dot-product attention、multi-head attention、decoder mask 相关的部分；
2. 用 SentencePiece 论文理解为什么工程上常用子词/无预分词 tokenizer；
3. 用 GPT-2 论文把“decoder-only + next-token likelihood + 大规模语料”放回语言模型历史；
4. 最后读本地 nanoGPT 的 `data/`、`model.py`、`train.py`、`sample.py`，以本阶段问题清单为导航。

阶段验收

- 不看资料画出“文本 → token ID → `(x,y)` → logits → loss/采样 → 新 token”的流程。
- 给定 `B=2,T=8,C=32,V=100`，能写出 embedding、attention 分数、logits 和交叉熵目标的典型形状。
- 能说明 causal mask 的上三角为什么必须被屏蔽。
- 能解释 temperature 不是“让模型重新训练”，top-k/top-p 不是改变模型参数。
- 能用一个小例子解释 KV cache 的内容、生命周期和内存代价。
- 能在 nanoGPT 中找到数据编码、模型前向、训练 loss 和生成循环的入口。
- 能运行极小模型并记录一次有依据的实验复盘，而不是只贴一段生成文本。

建议把“自测达到 8/10 + 三个核心代码程序运行通过 + 一页 nanoGPT 调用图”作为进入阶段 2 的最低标准。若 tokenizer、causal mask 或 KV cache 仍只能背 API，先复习对应学案，不要急着进入后训练。